

Creating Analysis Specification

Anthony G. Sena

2026-09-25

Contents

1	Creating an analysis specification	1
1.1	Setting up your R environment	1
1.2	Cohorts for the study	1
2	Assembling HADES modules	2
2.1	CohortGenerator Module Settings	2
2.2	CohortDiagnostics Module Settings	3
2.3	CohortIncidence Module Settings	3
2.4	Characterization Module Settings	4
2.5	CohortMethod Module Settings	4
2.6	SelfControlledCaseSeries Module Settings	6
2.7	PatientLevelPrediction Module Settings	8
3	Strategus Analysis Specifications	9

1 Creating an analysis specification

This walk through will show how to use **Strategus** to define an analysis specification on an example study using cohorts from the example problem *What is the risk of gastrointestinal (GI) bleed in new users of celecoxib compared to new users of diclofenac?* as described in the Book Of OHDSI Chapter 12 on Population Level Estimation

1.1 Setting up your R environment

Use **renv** and the **renv.lock** from the Strategus study template to set up your R environment. This is done by copying the **renv.lock** file into the root of a new project and the restore of the environment is done by calling **renv::restore()**. This will ensure that you have all of the R dependencies including the OHDSI HADES libraries and Strategus. The following code will download the **renv.lock** file to your machine, install **renv** and restore the R environment:

```
download.file("https://github.com/ohdsi-studies/StrategusStudyRepoTemplate/blob/main/renv.lock")
install.packages("renv")
renv::restore()
```

1.2 Cohorts for the study

To start, we'll need to define cohorts and negative control outcomes to use in our example analysis specification. We've included the cohorts and negative control outcomes in the **Strategus** package for this example and the code below will load them for use when assembling the analysis specification.

```

cohortDefinitionSet <- CohortGenerator::getCohortDefinitionSet(
  settingsFileName = "testdata/Cohorts.csv",
  jsonFolder = "testdata/cohorts",
  sqlFolder = "testdata/sql",
  packageName = "Strategus"
)
ncoCohortSet <- CohortGenerator::readCsv(file = system.file("testdata/negative_controls_concept_set.csv",
  package = "Strategus"
))

```

2 Assembling HADES modules

The building blocks of the **Strategus** analysis specification are HADES modules. For purposes of this walk through, a module is a specific analytic task you would like to perform. As shown in the subsequent sections, the high-level pattern for using a module consists of:

1. Instantiate the module object. For example, CohortGenerator's module is created using: `cg <- CohortGeneratorModule$new()`
2. Create the module specifications using the settings function(s) from the module. See the module list for more details.
3. Compose the analysis pipeline from one or more module settings.

2.1 CohortGenerator Module Settings

The following code instantiates a new CohortGenerator module `cgModule`. `cgModule` then exposes functions we can use for creating the module specifications to add to the analysis specification. In the analysis specification, we will add the cohort definitions and negative control outcomes to the `sharedResources` section since these elements may be used by any of the HADES modules. To do this, we will use the `createCohortSharedResourceSpecifications` and `createNegativeControlOutcomeCohortSharedResourceSpecifications` functions of the CohortGenerator module. In addition, we will use the `createModuleSpecifications` function to specify the cohort generation settings.

```

cgModule <- CohortGeneratorModule$new()

# Create the cohort definition shared resource element for the analysis specification
cohortDefinitionSharedResource <- cgModule$createCohortSharedResourceSpecifications(
  cohortDefinitionSet = cohortDefinitionSet
)

# Create the negative control outcome shared resource element for the analysis specification
ncoSharedResource <- cgModule$createNegativeControlOutcomeCohortSharedResourceSpecifications(
  negativeControlOutcomeCohortSet = ncoCohortSet,
  occurrenceType = "all",
  detectOnDescendants = TRUE
)

# Create the module specification
cohortGeneratorModuleSpecifications <- cgModule$createModuleSpecifications(
  generateStats = TRUE
)

```

2.2 CohortDiagnostics Module Settings

The following code creates the `cohortDiagnosticsModuleSpecifications` to run cohort diagnostics on the cohorts in the study.

```
cdModule <- CohortDiagnosticsModule$new()
cohortDiagnosticsModuleSpecifications <- cdModule$createModuleSpecifications(
  runInclusionStatistics = TRUE,
  runIncludedSourceConcepts = TRUE,
  runOrphanConcepts = TRUE,
  runTimeSeries = FALSE,
  runVisitContext = TRUE,
  runBreakdownIndexEvents = TRUE,
  runIncidenceRate = TRUE,
  runCohortRelationship = TRUE,
  runTemporalCohortCharacterization = TRUE
)
```

2.3 CohortIncidence Module Settings

The following code creates the `cohortIncidenceModuleSpecifications` to perform an incidence rate analysis for the target cohorts and outcome in this study.

```
ciModule <- CohortIncidenceModule$new()
targets <- list(
  CohortIncidence::createCohortRef(id = 1, name = "Celecoxib"),
  CohortIncidence::createCohortRef(id = 2, name = "Diclofenac"),
  CohortIncidence::createCohortRef(id = 4, name = "Celecoxib Age >= 30"),
  CohortIncidence::createCohortRef(id = 5, name = "Diclofenac Age >= 30")
)
outcomes <- list(CohortIncidence::createOutcomeDef(id = 1, name = "GI bleed", cohortId = 3, cleanWindow = 365))
tars <- list(
  CohortIncidence::createTimeAtRiskDef(id = 1, startWith = "start", endWith = "end"),
  CohortIncidence::createTimeAtRiskDef(id = 2, startWith = "start", endWith = "start", endOffset = 365)
)
analysis1 <- CohortIncidence::createIncidenceAnalysis(
  targets = c(1, 2, 4, 5),
  outcomes = c(1),
  tars = c(1, 2)
)
irDesign <- CohortIncidence::createIncidenceDesign(
  targetDefs = targets,
  outcomeDefs = outcomes,
  tars = tars,
  analysisList = list(analysis1),
  strataSettings = CohortIncidence::createStrataSettings(
    byYear = TRUE,
    byGender = TRUE
  )
)
cohortIncidenceModuleSpecifications <- ciModule$createModuleSpecifications(
  irDesign = irDesign$toList()
)
```

```
)
```

2.4 Characterization Module Settings

The following code creates the `characterizationModuleSpecifications` to perform an characterization analysis for the target cohorts and outcome in this study.

```
cModule <- CharacterizationModule$new()
characterizationModuleSpecifications <- cModule$createModuleSpecifications(
  targetIds = c(1, 2),
  outcomeIds = 3
)
```

2.5 CohortMethod Module Settings

The following code creates the `cohortMethodModuleSpecifications` to perform a comparative cohort analysis for this study.

```
cmModule <- CohortMethodModule$new()
negativeControlOutcomes <- lapply(
  X = ncoCohortSet$cohortId,
  FUN = CohortMethod::createOutcome,
  outcomeOfInterest = FALSE,
  trueEffectSize = 1,
  priorOutcomeLookback = 30
)

outcomesOfInterest <- lapply(
  X = 3,
  FUN = CohortMethod::createOutcome,
  outcomeOfInterest = TRUE
)

outcomes <- append(
  negativeControlOutcomes,
  outcomesOfInterest
)

tcos1 <- CohortMethod::createTargetComparatorOutcomes(
  targetId = 1,
  comparatorId = 2,
  outcomes = outcomes,
  excludedCovariateConceptIds = c(1118084, 1124300)
)

tcos2 <- CohortMethod::createTargetComparatorOutcomes(
  targetId = 4,
  comparatorId = 5,
  outcomes = outcomes,
  excludedCovariateConceptIds = c(1118084, 1124300)
)

targetComparatorOutcomesList <- list(tcos1, tcos2)

covarSettings <- FeatureExtraction::createDefaultCovariateSettings(addDescendantsToExclude = TRUE)
```

```

getDbCmDataArgs <- CohortMethod::createGetDbCohortMethodDataArgs(
  washoutPeriod = 183,
  firstExposureOnly = TRUE,
  removeDuplicateSubjects = "remove all",
  maxCohortSize = 100000,
  covariateSettings = covarSettings
)

createStudyPopArgs <- CohortMethod::createCreateStudyPopulationArgs(
  minDaysAtRisk = 1,
  riskWindowStart = 0,
  startAnchor = "cohort start",
  riskWindowEnd = 30,
  endAnchor = "cohort end"
)

matchOnPsArgs <- CohortMethod::createMatchOnPsArgs()
fitOutcomeModelArgs <- CohortMethod::createFitOutcomeModelArgs(modelType = "cox")
createPsArgs <- CohortMethod::createCreatePsArgs(
  stopOnError = FALSE,
  control = Cyclops::createControl(cvRepetitions = 1)
)

computeSharedCovBalArgs <- CohortMethod::createComputeCovariateBalanceArgs()
computeCovBalArgs <- CohortMethod::createComputeCovariateBalanceArgs(
  covariateFilter = FeatureExtraction::getDefaultTable1Specifications()
)

cmAnalysis1 <- CohortMethod::createCmAnalysis(
  analysisId = 1,
  description = "No matching, simple outcome model",
  getDbCohortMethodDataArgs = getDbCmDataArgs,
  createStudyPopArgs = createStudyPopArgs,
  fitOutcomeModelArgs = fitOutcomeModelArgs
)

cmAnalysis2 <- CohortMethod::createCmAnalysis(
  analysisId = 2,
  description = "Matching on ps and covariates, simple outcomeModel",
  getDbCohortMethodDataArgs = getDbCmDataArgs,
  createStudyPopArgs = createStudyPopArgs,
  createPsArgs = createPsArgs,
  matchOnPsArgs = matchOnPsArgs,
  computeSharedCovariateBalanceArgs = computeSharedCovBalArgs,
  computeCovariateBalanceArgs = computeCovBalArgs,
  fitOutcomeModelArgs = fitOutcomeModelArgs
)

cmAnalysisList <- list(cmAnalysis1, cmAnalysis2)

analysesToExclude <- NULL

cohortMethodModuleSpecifications <- cmModule$createModuleSpecifications(

```

```

cmAnalysisList = cmAnalysisList,
targetComparatorOutcomesList = targetComparatorOutcomesList,
analysesToExclude = analysesToExclude
)

```

2.6 SelfControlledCaseSeries Module Settings

The following code creates the `sccsModuleSpecifications` to perform a self-controlled case series analysis for this study.

```

sccsModule <- SelfControlledCaseSeriesModule$new()

# Exposures-outcomes -----
negativeControlOutcomeIds <- ncoCohortSet$cohortId
outcomeOfInterestIds <- c(3)
exposureOfInterestIds <- c(1, 2)

exposuresOutcomeList <- list()
for (exposureOfInterestId in exposureOfInterestIds) {
  for (outcomeOfInterestId in outcomeOfInterestIds) {
    exposuresOutcomeList[[length(exposuresOutcomeList) + 1]] <- SelfControlledCaseSeries::createExposure(
      outcomeId = outcomeOfInterestId,
      exposures = list(SelfControlledCaseSeries::createExposure(exposureId = exposureOfInterestId))
    )
  }
  for (negativeControlOutcomeId in negativeControlOutcomeIds) {
    exposuresOutcomeList[[length(exposuresOutcomeList) + 1]] <- SelfControlledCaseSeries::createExposure(
      outcomeId = negativeControlOutcomeId,
      exposures = list(SelfControlledCaseSeries::createExposure(exposureId = exposureOfInterestId, true))
    )
  }
}

# Analysis settings -----
getDbScCsDataArgs <- SelfControlledCaseSeries::createGetDbScCsDataArgs(
  studyStartDate = "",
  studyEndDate = "",
  maxCasesPerOutcome = 1e6,
  useNestingCohort = TRUE,
  nestingCohortId = 1,
  deleteCovariatesSmallCount = 0
)

createStudyPopulation6AndOlderArgs <- SelfControlledCaseSeries::createCreateStudyPopulationArgs(
  minAge = 18,
  naivePeriod = 365
)

covarPreExp <- SelfControlledCaseSeries::createEraCovariateSettings(
  label = "Pre-exposure",
  includeEraIds = "exposureId",
  start = -30,
  end = -1,
  endAnchor = "era start"
)

```

```

)

covarExposureOfInt <- SelfControlledCaseSeries::createEraCovariateSettings(
  label = "Main",
  includeEraIds = "exposureId",
  start = 0,
  startAnchor = "era start",
  end = 0,
  endAnchor = "era end",
  profileLikelihood = TRUE,
  exposureOfInterest = TRUE
)

calendarTimeSettings <- SelfControlledCaseSeries::createCalendarTimeCovariateSettings(
  calendarTimeKnots = 5,
  allowRegularization = TRUE,
  computeConfidenceIntervals = FALSE
)

seasonalitySettings <- SelfControlledCaseSeries::createSeasonalityCovariateSettings(
  seasonKnots = 5,
  allowRegularization = TRUE,
  computeConfidenceIntervals = FALSE
)

createSccsIntervalDataArgs <- SelfControlledCaseSeries::createCreateSccsIntervalDataArgs(
  eraCovariateSettings = list(covarPreExp, covarExposureOfInt),
  seasonalityCovariateSettings = seasonalitySettings,
  calendarTimeCovariateSettings = calendarTimeSettings,
  minCasesForTimeCovariates = 100000
)

fitSccsModelArgs <- SelfControlledCaseSeries::createFitSccsModelArgs(
  control = Cyclops::createControl(
    cvType = "auto",
    selectorType = "byPid",
    startingVariance = 0.1,
    seed = 1,
    resetCoefficients = TRUE,
    noiseLevel = "quiet"
  )
)

sccsAnalysis1 <- SelfControlledCaseSeries::createSccsAnalysis(
  analysisId = 1,
  description = "SCCS age 18-",
  getDbSccsDataArgs = getDbSccsDataArgs,
  createStudyPopulationArgs = createStudyPopulation6AndOlderArgs,
  createIntervalDataArgs = createSccsIntervalDataArgs,
  fitSccsModelArgs = fitSccsModelArgs
)

sccsAnalysisList <- list(sccsAnalysis1)

```

```
# SCCS module specs -----
sccsModuleSpecifications <- sccsModule$createModuleSpecifications(
  sccsAnalysisList = sccsAnalysisList,
  exposuresOutcomeList = exposuresOutcomeList,
  combineDataFetchAcrossOutcomes = FALSE
)
```

2.7 PatientLevelPrediction Module Settings

The following code creates the `plpModuleSpecifications` to perform a patient-level prediction analysis for this study.

```
plpModule <- PatientLevelPredictionModule$new()

makeModelDesignSettings <- function(targetId, outcomeId, popSettings, covarSettings) {
  invisible(PatientLevelPrediction::createModelDesign(
    targetId = targetId,
    outcomeId = outcomeId,
    restrictPlpDataSettings = PatientLevelPrediction::createRestrictPlpDataSettings(),
    populationSettings = popSettings,
    covariateSettings = covarSettings,
    preprocessSettings = PatientLevelPrediction::createPreprocessSettings(),
    modelSettings = PatientLevelPrediction::setLassoLogisticRegression(),
    splitSettings = PatientLevelPrediction::createDefaultSplitSetting(),
    runCovariateSummary = T
  ))
}

plpPopulationSettings <- PatientLevelPrediction::createStudyPopulationSettings(
  startAnchor = "cohort start",
  riskWindowStart = 1,
  endAnchor = "cohort start",
  riskWindowEnd = 365,
  minTimeAtRisk = 1
)

plpCovarSettings <- FeatureExtraction::createDefaultCovariateSettings()

modelDesignList <- list()
for (i in 1:length(exposureOfInterestIds)) {
  for (j in 1:length(outcomeOfInterestIds)) {
    modelDesignList <- append(
      modelDesignList,
      list(
        makeModelDesignSettings(
          targetId = exposureOfInterestIds[i],
          outcomeId = outcomeOfInterestIds[j],
          popSettings = plpPopulationSettings,
          covarSettings = plpCovarSettings
        )
      )
    )
  }
}
```

```
plpModuleSpecifications <- plpModule$createModuleSpecifications(
  modelDesignList = modelDesignList
)
```

3 Strategus Analysis Specifications

Finally, we will use the various shared resources and module specifications to construct the full set of analysis specifications and save it to the file system in JSON format.

```
analysisSpecifications <- createEmptyAnalysisSpecifications() %>%
  addSharedResources(cohortDefinitionSharedResource) %>%
  addSharedResources(ncoSharedResource) %>%
  addModuleSpecifications(cohortGeneratorModuleSpecifications) %>%
  addModuleSpecifications(cohortDiagnosticsModuleSpecifications) %>%
  addModuleSpecifications(cohortIncidenceModuleSpecifications) %>%
  addModuleSpecifications(characterizationModuleSpecifications) %>%
  addModuleSpecifications(cohortMethodModuleSpecifications) %>%
  addModuleSpecifications(sccsModuleSpecifications) %>%
  addModuleSpecifications(plpModuleSpecifications)
```

```
ParallelLogger::saveSettingsToJson(analysisSpecifications, file.path(params$analysisSettingsPath, param
```